

Infinite Locus – Full-Stack Readiness Assessment

Part A – Advanced Engineering Reasoning: Model Answers

A1. Concurrent React requests and state correctness

Problems in the original code

- Race condition / out-of-order responses: if the user changes filters quickly, an older request can resolve after a newer one and overwrite fresh data with stale data.
- No cancellation: fetch keeps running after query/page/category changes or after unmount, wasting bandwidth and risking setState-after-unmount warnings.
- No AbortController: rapid typing fires a new request on every keystroke with nothing to cancel the previous in-flight request.
- fetch does not reject on HTTP error status: only network failures hit .catch(); a 404/500 response is treated as success and JSON-parsed, producing corrupted UI state.
- React Strict Mode double-invokes effects in development; without a cleanup function, this causes duplicate network calls and double loading toggles.
- loading/error are not reset per request: an error from request #1 can remain visible after request #2 succeeds, or loading can flicker incorrectly.
- No debouncing on rapid input, so every keystroke triggers a network call instead of coalescing rapid changes.

Typed replacement

```
interface Student { id: string; name: string; category: string; }

type FetchStatus = 'idle' | 'loading' | 'success' | 'error';

interface StudentsState {
  status: FetchStatus;
  rows: Student[];
  error: string | null;
}

function useStudents(query: string, page: number, category: string, debounceMs = 300) {
  const [state, setState] = useState<StudentsState>({
    status: 'idle',
    rows: [],
    error: null,
  });

  useEffect(() => {
    const controller = new AbortController();
    let isCurrent = true; // guards against Strict Mode's discarded first pass

    const timer = setTimeout(async () => {
      setState(prev => ({ ...prev, status: 'loading', error: null }));
      try {
        const params = new URLSearchParams({ q: query, page: String(page), category });
        const res = await fetch(`/api/students?${params}`, { signal: controller.signal });

        if (!res.ok) {
```

```

        throw new Error(`Request failed with status ${res.status}`);
    }
    const data: Student[] = await res.json();

    if (isCurrent) {
        setState({ status: 'success', rows: data, error: null });
    }
} catch (err) {
    if ((err as Error).name === 'AbortError') return; // expected on cleanup, not an error
    if (isCurrent) {
        setState({ status: 'error', rows: [], error: (err as Error).message });
    }
}
}, debounceMs);

return () => {
    isCurrent = false;
    controller.abort();
    clearTimeout(timer);
};
}, [query, page, category, debounceMs]));

return state;
}

```

What this fixes

- AbortController cancels the in-flight request on every dependency change and on unmount, so no orphaned request can call setState late.
- The isCurrent flag additionally guards against a response that resolves in the tiny window before abort takes effect, and against Strict Mode's phantom first effect run.
- res.ok is checked explicitly so non-2xx responses go into the error branch instead of being parsed as success.
- A single FetchStatus union prevents loading and error from being true independently of each other.
- The 300ms setTimeout debounces rapid typing so a burst of keystrokes produces one request, not one per character.

Protections still required on the server

- Server-side pagination and input-length limits — a client can still bypass debounce (e.g., via a scripted client) and hammer the endpoint.
- Rate limiting / throttling per user or IP, independent of client behavior.
- Query parameter validation and sanitization to prevent SQL/NoSQL injection through q and category.
- Authorization/tenant scoping on the server — the client cancelling a request is not a security control.
- Reasonable query timeout on the server so an abandoned client connection does not tie up a database connection indefinitely; ideally listen for the request's abort signal server-side (e.g., Node's req.on('close')) to cancel the underlying DB query.

A2. TypeScript state model with impossible states

A discriminated union keys every variant on a status field and only attaches the fields that are valid for that status, so the compiler rejects states like "loading with a stale error" or "success with no data."

Discriminated union and reducer

```
interface StudentDetails { id: string; name: string; scores: Record<string, number>; }
type RequestId = string;

type StudentDetailsState =
  | { status: 'idle' }
  | { status: 'loading'; requestId: RequestId; previousData?: StudentDetails }
  | { status: 'success'; requestId: RequestId; data: StudentDetails }
  | { status: 'error'; requestId: RequestId; message: string; staleData?: StudentDetails };

type Action =
  | { type: 'FETCH_START'; requestId: RequestId }
  | { type: 'FETCH_SUCCESS'; requestId: RequestId; data: StudentDetails }
  | { type: 'FETCH_ERROR'; requestId: RequestId; message: string };

function currentRequestId(state: StudentDetailsState): RequestId | undefined {
  return state.status === 'idle' ? undefined : state.requestId;
}

function reducer(state: StudentDetailsState, action: Action): StudentDetailsState {
  switch (action.type) {
    case 'FETCH_START': {
      const previousData =
        state.status === 'success' ? state.data
        : state.status === 'error' ? state.staleData
        : undefined;
      return { status: 'loading', requestId: action.requestId, previousData };
    }

    case 'FETCH_SUCCESS': {
      if (action.requestId !== currentRequestId(state)) return state; // ignore stale/out-of-
order response
      return { status: 'success', requestId: action.requestId, data: action.data };
    }

    case 'FETCH_ERROR': {
      if (action.requestId !== currentRequestId(state)) return state;
      const staleData = state.status === 'loading' ? state.previousData : undefined;
      return { status: 'error', requestId: action.requestId, message: action.message, staleData };
    }
  };
}

default: {
  const exhaustiveCheck: never = action;
  return exhaustiveCheck;
}
}
```

Behaviour on a failed refresh

- Before refetching, FETCH_START carries the currently visible data forward as previousData so the UI never blanks out during a background refresh.

- If the refresh fails, `FETCH_ERROR` moves that carried-forward data into `staleData` on the error variant — the component can render "showing last known data, refresh failed" plus a retry action, using the same field name (`staleData`) so the render logic only has one place to read it from.
- Every action carries the `requestId` it was issued for; the reducer discards any `FETCH_SUCCESS/FETCH_ERROR` whose `requestId` does not match the state's current `requestId`, which is what makes out-of-order responses harmless without needing an `AbortController` inside the reducer itself.
- Because the type is a union, e.g. `state.status === 'success'` narrows `state.data` to be defined and `state.error` to not exist at all — "loading with a stale error" simply has no matching variant and cannot be constructed.

A3. Idempotency and optimistic concurrency

HTTP contract

- `POST /assessments` requires a client-generated Idempotency-Key header (UUID v4) representing one logical create attempt.
- `PATCH /assessments/:id` requires the client's last-known version (body field `expectedVersion`, or an `If-Match` header) representing optimistic concurrency control.

Status codes

- 201 Created — first successful processing of an Idempotency-Key.
- 200 OK — replayed response for a key that was already processed (retry).
- 409 Conflict — version mismatch on `PATCH` (two evaluators editing concurrently), or an Idempotency-Key reused with a different payload.
- 422 Unprocessable Entity — payload fails validation.
- 400 Bad Request — Idempotency-Key header missing on `POST`.

Persistence behaviour

- Idempotency keys are stored in a table (`key`, `tenant_id`, `request_hash`, `response_status`, `response_body`, `created_at`, `expires_at`) with a unique constraint on (`key`, `tenant_id`).
- On `POST`, the handler looks up the key inside the same transaction that inserts the assessment. If found with a matching request hash, it replays the stored response without touching business data again. If found with a different hash, it is a client bug (key reuse) and returns 409/422.
- Storage lifetime: keys are retained roughly 24–48 hours — long enough to cover retry storms from flaky mobile connections — then expired via a TTL index or scheduled cleanup job.
- Concurrent edits use a version column on the assessment row. The update is a single conditional statement: `UPDATE ... SET version = version + 1 WHERE id = ? AND version = ?`. Zero rows affected means someone else updated first, so the server returns 409 with the current row for the client to re-merge.
- What must happen inside one database transaction: the assessment insert/update and the idempotency-key write (or the version check) must commit or roll back together — otherwise a crash between the two steps could let a retry either duplicate the record or return a stale replay.

Implementation sketch

```

async function createAssessment(req: Request, res: Response) {
  const idempotencyKey = req.header('Idempotency-Key');
  if (!idempotencyKey) {
    return res.status(400).json({ error: 'Idempotency-Key header is required' });
  }
  const requestHash = hash(req.body);

  return db.transaction(async (trx) => {
    const existing = await trx('idempotency_keys')
      .where({ key: idempotencyKey, tenant_id: req.tenantId })
      .first();

    if (existing) {
      if (existing.request_hash !== requestHash) {
        return res.status(409).json({ error: 'Idempotency key reused with a different payload'
});
      }
      return res.status(existing.response_status).json(existing.response_body);
    }

    const [assessment] = await trx('assessments')
      .insert({ ...req.body, tenant_id: req.tenantId, version: 1 })
      .returning('*');

    await trx('idempotency_keys').insert({
      key: idempotencyKey,
      tenant_id: req.tenantId,
      request_hash: requestHash,
      response_status: 201,
      response_body: assessment,
      expires_at: new Date(Date.now() + 24 * 60 * 60 * 1000),
    });

    return res.status(201).json(assessment);
  });
}

async function updateAssessment(req: Request, res: Response) {
  const { expectedVersion, ...changes } = req.body;

  const updated = await db('assessments')
    .where({ id: req.params.id, tenant_id: req.tenantId, version: expectedVersion })
    .update({ ...changes, version: db.raw('version + 1'), updated_at: new Date() })
    .returning('*');

  if (updated.length === 0) {
    const current = await db('assessments').where({ id: req.params.id }).first();
    return res.status(409).json({ error: 'Version conflict, refresh and retry', current });
  }
  return res.status(200).json(updated[0]);
}

```

A4. SQL: latest evidence, weighted score, and ties

```

WITH latest_attempts AS (
  SELECT DISTINCT ON (a.student_id, a.competency_id)
    a.student_id, a.competency_id, a.score
  FROM attempts a

```

```

ORDER BY a.student_id, a.competency_id, a.submitted_at DESC, a.id DESC
-- id DESC is the deterministic tiebreaker when submitted_at is equal
),
weighted AS (
  SELECT
    s.id AS student_id,
    SUM(COALESCE(la.score, 0) * cw.weight) / NULLIF(SUM(cw.weight), 0) AS calculated_score
  FROM students s
  CROSS JOIN competencies c
  JOIN competency_weights cw ON cw.competency_id = c.id
  LEFT JOIN latest_attempts la
    ON la.student_id = s.id AND la.competency_id = c.id
  GROUP BY s.id
)
SELECT s.id, s.name, w.calculated_score, s.current_score
FROM students s
JOIN weighted w ON w.student_id = s.id
WHERE w.calculated_score IS DISTINCT FROM s.current_score;

```

Reasoning

- `DISTINCT ON (student_id, competency_id)` with `ORDER BY submitted_at DESC, id DESC` keeps exactly one row per student/competency — the newest submission, and when two attempts share a timestamp, the higher id (the one inserted later) wins deterministically.
- `CROSS JOIN` competencies against every student, then `LEFT JOIN` the latest attempt, guarantees every student × competency pair exists even when no attempt was ever submitted; `COALESCE(la.score, 0)` turns a missing attempt into a zero contribution rather than dropping the row.
- `IS DISTINCT FROM` (not `!=`) is used in the final filter because it correctly treats `NULL` vs a value as "different", whereas `!=` would silently exclude rows where either side is `NULL`.

Indexes

- `attempts(student_id, competency_id, submitted_at DESC, id DESC)` — a composite index that lets the `DISTINCT ON` scan walk straight to the latest row per group without a sort.
- `competency_weights(competency_id)` — supports the join to weights (typically already the primary key).

Isolation concern

Under `READ COMMITTED`, a new attempt could be inserted between the CTE that computes `latest_attempts` and the outer comparison against `students.current_score`, so the query might mix a stale `current_score` with a freshly changed "latest" attempt from a concurrent transaction. Running this as a single statement inside a `REPEATABLE READ` (or `SERIALIZABLE`) transaction gives it one consistent snapshot for the whole calculation, so results are not skewed by writes that commit mid-query.

A5. Transactional race condition

Two requests both read the aggregate score as 72, each computes a new total in application code, and the second `UPDATE` overwrites the first — a classic lost update. Both attempt rows exist, but the aggregate reflects only one of them.

Strategy 1 — Pessimistic locking (SELECT ... FOR UPDATE)

Lock the aggregate row before reading it, so the second transaction blocks until the first commits and then reads the updated value. Simple and correct, but serializes all writers on that row and can deadlock if lock order is not fixed across tables.

Strategy 2 — Atomic increment (no read-modify-write)

Never read the current value into application code at all; let the database compute the new value in one statement (UPDATE aggregate SET total = total + \$delta). This removes the read-modify-write gap entirely and needs no explicit lock, since the row-level update is atomic. This is the strategy to ship, combined with an idempotency guard so a client retry cannot double-apply the same attempt.

Comparison

- FOR UPDATE: strong correctness guarantee, but holds a row lock for the duration of the transaction — higher contention under load and deadlock risk if multiple rows are locked in inconsistent order across requests.
- Atomic increment: no explicit lock held by the application; the database's own row-level write lock is momentary. Scales better under concurrent writers and has no deadlock risk from this pattern alone.
- Optimistic (version check, as in A3) is a third option — best when conflicts are rare, since it avoids locking entirely but requires client-side retry logic on 409.

Pseudocode for the shipped strategy

```
BEGIN;

-- 1. Insert the attempt idempotently, keyed by a client-supplied request id
INSERT INTO attempts (student_id, competency_id, score, submitted_at, client_request_id)
VALUES ($student_id, $competency_id, $score, now(), $client_request_id)
ON CONFLICT (client_request_id) DO NOTHING
RETURNING id;

-- 2. Only apply the aggregate delta if step 1 actually inserted a new row
-- (checked in application code from the RETURNING result set)
UPDATE student_aggregates
SET total_score = total_score + $score,
    attempt_count = attempt_count + 1
WHERE student_id = $student_id;

COMMIT;
```

A unique constraint on client_request_id makes step 1 a no-op on retry; the application only runs step 2 when step 1's RETURNING set is non-empty, so a network retry that resends the same request cannot double-count the aggregate even though the client saw a timeout and tried again.

A6. MongoDB aggregation and anomaly detection

```
db.activity_events.aggregate([
  { $match: { occurredAt: { $gte: new Date(Date.now() - 24 * 60 * 60 * 1000) } } },
  // Roll up per tenant + assessment first, so duplicate/retry events collapse
```

```

{ $group: {
  _id: { tenantId: "$tenantId", assessmentId: "$assessmentId" },
  successCount: { $sum: { $cond: [{ $eq: ["$eventType", "success"] }, 1, 0] } },
  failureCount: { $sum: { $cond: [{ $eq: ["$eventType", "validation_failure"] }, 1, 0] } },
  eventCount: { $sum: 1 },
  latencies: {
    $push: {
      $cond: [{ $eq: ["$eventType", "success"] }, "$metadata.latencyMs", "$$REMOVE"]
    }
  }
}},

// Then roll up per tenant
{ $group: {
  _id: "$_id.tenantId",
  uniqueSuccessfulAssessments: { $sum: { $cond: [{ $gte: ["$successCount", 1] }, 1, 0] } },
  multiSuccessAssessments: { $sum: { $cond: [{ $gt: ["$successCount", 1] }, 1, 0] } },
  totalFailures: { $sum: "$failureCount" },
  totalEvents: { $sum: "$eventCount" },
  allLatencies: { $push: "$latencies" }
}},

{ $addFields: {
  allLatencies: { $reduce: { input: "$allLatencies", initialValue: [], in: { $concatArrays:
["$$value", "$$this"] } } }
}},
{ $addFields: { sortedLatencies: { $sortBy: { input: "$allLatencies", sortBy: 1 } } } },
{ $addFields: {
  p95SubmissionLatencyMs: {
    $let: {
      vars: { n: { $size: "$sortedLatencies" } },
      in: {
        $cond: [
          { $eq: ["$$n", 0] },
          null,
          { $arrayElemAt: ["$sortedLatencies", { $floor: { $multiply: [0.95, { $subtract:
["$$n", 1] } ] } } ] }
        ]
      }
    }
  }
}},

{ $project: {
  _id: 0,
  tenantId: "$_id",
  uniqueSuccessfulAssessments: 1,
  multiSuccessAssessments: 1,
  validationFailureRate: {
    $cond: [{ $eq: ["$totalEvents", 0] }, 0, { $divide: ["$totalFailures", "$totalEvents"] } ]
  },
  p95SubmissionLatencyMs: 1
}}
]);

```

Indexes

- { tenantId: 1, occurredAt: -1 } — supports the initial \$match plus per-tenant grouping.
- { assessmentId: 1, eventType: 1, occurredAt: -1 } — supports efficiently finding all events for an assessment when investigating the "more than one success" anomaly flagged by the pipeline.

Why counting documents alone is unsafe

- At-least-once delivery and client retries mean the same logical success can be recorded as two or three separate events for one assessment; counting raw documents overstates unique successful assessments.
- The pipeline above groups by (tenantId, assessmentId) first and derives uniqueSuccessfulAssessments from whether successCount >= 1, not from a raw document count — this is exactly what also lets it flag multiSuccessAssessments (successCount > 1) as a genuine anomaly worth investigating (duplicate submission, replay bug, or a client double-fire) rather than silently inflating throughput metrics.

A7. Security review

Vulnerabilities identified

- Mass assignment: spreading the raw request body into the ORM's create() call lets the client set any column, including ones it should never control.
- Client-supplied tenantId: allows a request to write data into, or read data from, a tenant the caller does not belong to — a direct broken-access-control / IDOR issue.
- Client-supplied evaluatorRole: the server trusts a role claim sent in the body instead of deriving it from the authenticated session, enabling privilege escalation (a student-level caller can claim to be an evaluator).
- Stored XSS: rendering notes as raw HTML means any script tag or event handler a caller submits executes in the browser of whoever later views that assessment — including admins, which can lead to session/token theft.
- No schema validation: score, category and other fields are trusted as-is, allowing out-of-range scores, invalid categories, or oversized payloads.
- No audit trail: there is no record of who created or changed a given assessment, which blocks incident investigation and non-repudiation.

Top 3 risks, ranked, with fixes

- 1. Broken access control via client-supplied tenantId and mass assignment:** Highest risk — a caller can write into or exfiltrate another tenant's data, which is a full multi-tenant data breach. Fix: never spread the request body into the create call; use an explicit allow-list of fields, and derive tenantId strictly from the authenticated session/JWT, never from the body.
- 2. Stored XSS from unsanitized notes:** Second highest — persistent script execution in other users' (including admins') browsers, enabling session hijacking. Fix: sanitize on write with an allow-list sanitizer (e.g., DOMPurify server-side with allowedTags: []), and render with an auto-escaping templating layer or React's default text rendering rather than dangerouslySetInnerHTML.
- 3. Client-supplied evaluatorRole enabling privilege escalation:** Fix: derive the caller's role from the authenticated identity/session (or a role lookup against the database), never trust a role field in the request body; add an explicit authorize() middleware check before the handler runs.

Corrected handler

```
const CreateAssessmentSchema = z.object({
```

```

studentId: z.string().uuid(),
score: z.number().min(0).max(100),
category: z.enum(['quiz', 'project', 'exam']),
notes: z.string().max(2000),
});

app.post('/assessments', authenticate, authorize('evaluator'), async (req, res) => {
  const parsed = CreateAssessmentSchema.parse(req.body);

  const tenantId = req.user.tenantId;      // from verified session, never from body
  const evaluatorId = req.user.id;

  const assessment = await db.transaction(async (trx) => {
    const [created] = await trx('assessments').insert({
      studentId: parsed.studentId,
      score: parsed.score,
      category: parsed.category,
      notes: sanitizeHtml(parsed.notes, { allowedTags: [], allowedAttributes: {} }),
      tenantId,
      evaluatorId,
    }).returning('*');

    await trx('audit_log').insert({
      actor: evaluatorId,
      action: 'assessment.create',
      entityId: created.id,
      tenantId,
      payload: parsed,
      at: new Date(),
    });

    return created;
  });

  res.status(201).json(assessment);
});

```

A8. Tests that detect real failures

Property-based invariant — weighted score calculation

```

import fc from 'fast-check';

test('weighted score is always within [0, 100] and matches the formula', () => {
  fc.assert(
    fc.property(
      fc.array(
        fc.record({ score: fc.integer({ min: 0, max: 100 }), weight: fc.integer({ min: 1, max: 10 }) })
      ),
      { minLength: 1 }
    ),
    (attempts) => {
      const result = calculateWeightedScore(attempts);
      const expected = attempts.reduce((s, a) => s + a.score * a.weight, 0)
        / attempts.reduce((s, a) => s + a.weight, 0);
      expect(result).toBeGreaterThanOrEqual(0);
      expect(result).toBeLessThanOrEqual(100);
      expect(result).toBeCloseTo(expected, 5);
    }
  )
});

```

```
);
});
```

Integration test — idempotent retry against a real database

```
test('retrying POST /assessments with the same Idempotency-Key does not duplicate the row', async () => {
  const key = crypto.randomUUID();
  const payload = { studentId, score: 88, category: 'quiz', notes: 'good work' };

  const first = await request(app).post('/assessments')
    .set('Idempotency-Key', key).send(payload);
  const second = await request(app).post('/assessments')
    .set('Idempotency-Key', key).send(payload);

  expect(first.status).toBe(201);
  expect(second.status).toBe(200);
  expect(second.body.id).toBe(first.body.id);

  const rows = await db('assessments').where({ studentId });
  expect(rows).toHaveLength(1);
});
```

Component test — stale React responses

```
test('only the latest search results are rendered when typing quickly', async () => {
  server.use(
    rest.get('/api/students', async (req, res, ctx) => {
      const q = req.url.searchParams.get('q');
      const delay = q === 'ann' ? 200 : 20; // simulate the first request resolving last
      await new Promise(r => setTimeout(r, delay));
      return res(ctx.json([ { id: '1', name: `result-for-${q}` } ]));
    })
  );

  render(<SearchStudents />);
  const input = screen.getByRole('textbox');
  await userEvent.type(input, 'ann');
  await userEvent.clear(input);
  await userEvent.type(input, 'bob');

  await waitFor(() => {
    expect(screen.getByText('result-for-bob')).toBeInTheDocument();
    expect(screen.queryByText('result-for-ann')).not.toBeInTheDocument();
  });
});
```

Failure-injection test — database rollback

```
test('a crash inside the transaction leaves no partial write', async () => {
  const before = await db('student_aggregates').where({ studentId }).first();

  jest.spyOn(db, 'transaction').mockImplementationOnce(async (fn) => {
    return db.transaction(async (trx) => {
      await trx('attempts').insert({ studentId, score: 90 });
      throw new Error('simulated crash before aggregate update');
    });
  });
});
```

```

await expect(submitAttempt({ studentId, score: 90 })).rejects.toThrow();

const after = await db('student_aggregates').where({ studentId }).first();
const attempts = await db('attempts').where({ studentId, score: 90 });

expect(after.total_score).toBe(before.total_score); // aggregate unchanged
expect(attempts).toHaveLength(0); // insert was rolled back too, same transaction
});

```

Additional tests worth naming in an interview

- Concurrent submission test: fire two PATCH requests in parallel with the same expectedVersion and assert exactly one returns 200 and the other returns 409.
- Authorization test: call an endpoint with a valid token for tenant A against a resource owned by tenant B and assert 403/404, never 200.

What not to mock, and why

- The database: use a real Postgres instance (e.g., via testcontainers) for concurrency, transaction-rollback and isolation-level tests — a mocked ORM cannot reproduce real row locks, unique-constraint conflicts, or MVCC snapshot behaviour, so those tests would pass even if the real locking logic were wrong.
- The ORM's transaction boundary — mocking trx would hide exactly the bug class A5's race condition is about.
- What is fine to mock: third-party services (email/SMS providers), the system clock (for TTL/expiry tests), and any outbound webhook calls — these are external to the correctness being tested.

A9. Git recovery and AI verification

Safe team recovery procedure

1. Treat it as a security incident first, not just a git problem: assess and contain the cross-tenant leak (who was exposed, for how long) in parallel with the git recovery, independent of whether history gets fixed.
2. Stop everyone from pushing to or pulling the affected branch until a safe tip is restored, to avoid the leak (or a botched fix) spreading further.
3. On the developer's machine, recover the pre-leak commit from git reflog show — reflog is local-only and expires (commonly ~90 days), so this must happen before any git gc prunes it.
4. Verify the recovered commit is actually safe with git show <sha>, then immediately tag and push it: git tag recovery/pre-leak <sha> && git push origin recovery/pre-leak — this preserves the evidence trail permanently instead of leaving it in a perishable reflog.
5. Before any further destructive operation, snapshot everything for the post-mortem: git bundle create incident.bundle --all and export the reflog to a file.
6. Fix forward rather than force-pushing again: branch from the safe commit and use git revert on the offending commit(s) on top of the current main, which removes the leaking code while keeping full history intact for audit — avoid a second force-push, which would hide what happened.

7. If history genuinely must be rewritten (e.g., a secret was committed, not just buggy code), coordinate a scheduled team-wide re-clone/reset window with explicit instructions, rather than letting each developer merge their local state back in and silently resurrect the bad commits.

Checks before approving the corrected AI-assisted patch

- Mandatory line-by-line review by a second engineer — AI-suggested code is treated as an untrusted first draft, never merged on the strength of "it looks right".
- A regression test that specifically encodes the leak (e.g., "tenant A's token cannot read tenant B's assessment") must be added and must fail on the old code / pass on the new code.
- Automated secret scanning (e.g., gitleaks) run across the patch and, given the incident, across recent history.
- Explicit check that the fix doesn't reintroduce the same class of trust-boundary bug — e.g., confirm `tenantId` is still derived from the session, not from client input, per the fix pattern in A7.
- Full CI green, including the authorization/integration tests from A8, before merge.
- Security sign-off given the incident involved actual data exposure, not just a code-review approval.

PART-B Link:- <https://github.com/garima-m14/student-readiness-control-center>

PART-C Link:- https://github.com/garima-m14/production_incident_investigation-